Foveated Screen Perception: Bio-Inspired Top-Down Observation Control for Computer-Use Agents

Xiaoyong Liang
xiaoyong@utexas.edu

April 2026

Abstract

Computer-use agents built on vision-language models send a full screenshot to the model at every step, consuming approximately 1,300 tokens per observation. A 50-step desktop task therefore spends roughly 65,000 tokens on screenshots alone, often exceeding 80% of total context, yet recent evidence shows this visual history does not improve task performance: only text-based action history helps [48]. Agents also take 1.4–2.7× more steps than necessary, with GUI grounding errors as the dominant failure mode.

We introduce **Foveated Screen Perception** (FSP), a bio-inspired observation interface that replaces the fixed screenshot-per-step loop with an `observe` tool whose three modes map one-to-one onto the three parallel streams the primate retina sends to cortex: `scan` (rods + magnocellular + dorsal: low-resolution grayscale image of the full screen with change annotations, ∼200 tokens), `focus` (cones + parvocellular + ventral: high-resolution full-color crop at specified coordinates, ∼150 tokens), and `read` (Visual Word Form Area: symbolic text extraction at specified coordinates, ∼40 tokens). The decomposition is not a stylistic choice; it is the same compression strategy the retina uses to reduce $\sim 10^9$ bits/sec of photoreceptor input to $\sim 10^7$ bits/sec at cortex, achieved by shedding color and resolution in the wide-field stream, paying for fidelity only at the fovea, and bypassing pixels entirely for text. The LLM selects its observation mode at each step based on current task goals, a direct analog of top-down attentional control in the primate visual system, where high-level objectives bias which sensory stream is consulted.

We evaluate FSP in a 2×2 factorial design crossing two frontier models (GPT-5.4 and Claude Opus 4.6) with two perception conditions (standard full-screenshot and FSP) on the ScreenSpot-Pro grounding benchmark and OSWorld desktop task suite. FSP is designed to yield an estimated 10–13× reduction in observation tokens per task while targeting comparable or improved grounding accuracy and task completion. We release the `observe` tool implementation, evaluation harness, and all experimental logs.

1 Introduction

The dominant architecture for computer-use agents is uniform. Whether the system is OpenAI Operator [33], Google Mariner [13], ByteDance UI-TARS [35, 34], or Anthropic's Claude Computer Use [2], the perception loop is the same: capture a full screenshot, encode it as tokens, send it to a vision-language model, receive an action, execute, and repeat. Every step pays the same cost, roughly 1,300 tokens on Claude and 765–1,105 on GPT-4o, regardless of whether the screen changed, whether the agent needs visual information at all, or whether a few words of text would suffice.

This is expensive. A routine 50-step desktop task burns approximately 65,000 tokens on screenshot encoding alone, and screenshot tokens typically constitute 80% or more of total context. The cost extends beyond token usage. The growing visual context actively degrades performance. The OSWorld-Human study [48] found that planning and reflection calls consume 75–94% of total task latency, with each successive step taking up to 3× longer as context grows.

The study also showed that screenshot-only history does not improve performance; only text-based action history helps. Agents routinely take 1.4–2.7× more steps than human-optimal trajectories, and across OSWorld [49], ScreenSpot-Pro [21], and WindowsAgentArena [4], the dominant failure mode is GUI grounding errors: misclicking, selecting the wrong element, or failing to locate a target that occupies a mere 0.07% of the viewport.

1.1 How Human Vision Solves the Bandwidth Problem

The primate visual system solves an analogous bandwidth problem through a specific architectural trick: the retina does not send one signal to cortex, it sends at least three parallel streams, each optimized for a different kind of information and each with a characteristic bandwidth [**?**, 18]. Rods (∼120 million cells) drive a low-resolution grayscale wide-field stream (magnocellular, dorsal, "where") optimized for spatial layout, motion, and luminance contrast; they achieve their sensitivity by pooling up to 100 photoreceptors onto a single ganglion cell, at the cost of resolution and color [**?**]. Cones (∼6 million cells) drive a high-resolution color stream (parvocellular, ventral, "what") optimized for object identity and fine detail, but concentrate almost all of their bandwidth on the ∼2° foveal center [42, 12]. A specialized patch of left fusiform cortex, the Visual Word Form Area (VWFA), extracts symbolic text from the ventral stream, bypassing pixel-level processing for the specific case of reading [**?**]. Together these streams compress $\sim 10^9$ bits/sec of retinal input to $\sim 10^7$ bits/sec at cortex, a 100× reduction achieved before any conscious processing begins [18]. On top of this retinal decomposition sit further mechanisms: gist extraction (scene categorization within 36–100ms before any saccade [32, 41]), saccadic targeting (3–4 goal-directed fixations per second [19]), top-down attentional control (prefrontal goal representations bias visual processing before the stimulus arrives [9, 47]), and predictive coding (only prediction errors propagate upward through the cortical hierarchy [36]). Current computer-use agents implement none of them.

1.2 The Text Insight

A critical but underappreciated fact about screen interaction is that the vast majority of it is text comprehension. When a user reads a menu, checks an error message, scans a form field label, or verifies a status indicator, the visual processing is merely a means to an end: the information being extracted is text. Eye-tracking studies of knowledge workers show that over 70% of fixation time is spent on textual elements [37].

This suggests a fundamental inefficiency in the screenshot-per-step paradigm. Encoding a 1920×1080 screenshot as ∼1,300 vision tokens in order to read a 5-word error message is a 100× overhead in token cost for no informational gain; the text could be delivered directly via OCR or accessibility APIs at a cost of 10–20 tokens. The field has converged on this insight from multiple directions: Fara-7B [44] retains only the last 3 screenshots but keeps all text history; the JetBrains "Complexity Trap" study [3] found that replacing old observations with text placeholders halves cost with no performance loss; and DirectShell [24] achieves 85% task success using only accessibility-tree text at 50–200 tokens per step (though it fails on applications with incomplete accessibility support [27]).

1.3 Prior Work on Targeted Observation

The zoom-and-crop literature has shown that letting agents inspect sub-regions of the screen consistently improves grounding accuracy, often without retraining. RegionFocus [20] improves UI-TARS-72B on ScreenSpot-Pro from 38.1% to 50.2% via iterative multi-aspect-ratio crops. ZoomClick [53] achieves 73.1% with a training-free 3-stage zoom cascade. AdaZoom-GUI [46] reaches 76.8% using GRPO-trained conditional zoom on small predicted elements. GUI-Eyes

[5] is the first system to use RL to train the zoom decision itself, achieving 44.8% with a 3B model that learns when to crop and when not to.

These systems share two limitations. First, they operate exclusively in the image domain: every observation mode returns pixels, whether a full screenshot, a crop, or a zoomed region, despite the evidence that text extraction is sufficient for most interaction steps. Second, and more fundamentally, they treat the screenshot as one monolithic modality: full RGB color, uniform resolution, the whole viewport at once. The retina, solving a structurally identical bandwidth problem, does not do this – it explicitly separates a cheap grayscale wide-field stream (magnocellular) from an expensive color foveal stream (parvocellular). No existing GUI agent system exposes this distinction as an agent-selectable tool. Claude Computer Use's `zoom` action [2] is the closest production feature, but it returns a full-color crop and offers no coarser or cheaper alternative.

1.4 Foveated Screen Perception

We propose **Foveated Screen Perception** (FSP), an observation interface that gives the LLM explicit control over which retinal stream it samples. FSP provides a single `observe` tool with three modes mapped one-to-one onto the parallel streams of primate vision: `scan` (low-resolution grayscale full-screen view with change annotations, analogous to rods / magnocellular / dorsal), `focus` (high-resolution full-color crop at specified coordinates, analogous to cones / parvocellular / ventral), and `read` (symbolic text extraction, analogous to VWFA). The agent selects its mode based on current task goals, implementing top-down attentional control over which stream is consulted. We describe the full design in Section 3.

1.5 Contributions

1. We introduce the `observe` tool interface, the first GUI-agent perception system whose modes map directly onto the three parallel streams of the primate retina (magnocellular wide-field grayscale, parvocellular foveal color, and VWFA text extraction).
2. We argue that the grayscale-vs-color choice is as fundamental to efficient observation as the crop-vs-full-frame choice that the existing zoom/crop literature has focused on, and we expose both axes as agent-facing tool parameters.
3. We frame LLM observation control as top-down attentional selection among retinal streams, and identify the missing bottom-up (stimulus-driven) component as a concrete direction for future work.
4. We evaluate FSP in a 2×2 factorial design crossing two frontier models (GPT-5.4 and Claude Opus 4.6) with two perception conditions on ScreenSpot-Pro and OSWorld, controlling for the possibility that results are model-specific.
5. We project an estimated 10–13$\times$ reduction in observation tokens per task, with a typical step costing 40 tokens (`read`) or 200 tokens (`scan`) versus 1,300 tokens (full screenshot).

2 Related Work

FSP sits at the intersection of several active research threads. These include how computer-use agents observe their environment, how zoom and crop improve GUI grounding, the tradeoff between accessibility trees and vision, context compression methods, and bio-inspired visual processing. We survey each in turn, highlighting the gap that motivates our unified observation-control framework.

2.1 Observation in Computer-Use Agents

Nearly every production computer-use system follows the same perception loop: capture a full screenshot, send it to a vision-language model (VLM), receive an action, execute, and repeat. OpenAI Operator and its underlying Computer-Using Agent (CUA) API [33], Google Mariner [13], ByteDance's UI-TARS and UI-TARS-2 [35, 34], and SeeAct-V [52] all employ this pattern. The model receives the full viewport at every turn with no crop, zoom, or modality selection, a design inherited from early web agents rather than derived from any analysis of what information the model actually needs.

Two notable exceptions depart from this paradigm. **Claude Computer Use** [2] exposes a `zoom` action that returns a cropped, upscaled sub-region of the screen. This is the only production API where the model itself can request a targeted view, and Claude's OSWorld score reached approximately 72.5% in late 2025, roughly double the previous state of the art. **GUI-Eyes** [5] goes further by RL-training the agent to decide whether and how to invoke perception tools (`crop` and `zoom`), using a composite reward $R = 0.6R_{\text{acc}} + 0.1R_{\text{format}} + 0.3R_{\text{tool}}$ where R_{tool} combines spatial proximity and region IoU. A 3B-parameter model with GUI-Eyes reaches 44.8% on ScreenSpot-Pro, compared to 30.2% without, a 48% relative improvement from perception control alone.

Despite these advances, no existing system provides the agent with a unified observation interface that spans multiple modalities (text extraction, scene gist, image crop, change verification) and defaults to text-only output.

2.2 Zoom and Crop for GUI Grounding

The small-element problem is the core driver of grounding errors in screen understanding. ScreenSpot-Pro targets occupy only 0.07% of total image area [21], compared to 2.01% in older benchmarks, making precise localization of UI elements a severe challenge for VLMs processing full-resolution screenshots. A rapidly growing body of work demonstrates that letting agents zoom into candidate regions yields dramatic grounding improvements, often with no retraining required.

RegionFocus [20], presented at ICCV 2025, performs iterative zoom with multiple aspect-ratio crops and stamps pink-star markers at prior attempt locations to prevent revisiting. It improves UI-TARS-72B from 38.1% to 50.2% and Qwen2.5-VL-72B from 47.8% to 61.6% on ScreenSpot-Pro. **ScreenSeeker** [51] uses GPT-4o as a planner to guide cascaded recursive cropping, with a smaller OS-Atlas-7B model grounding within each patch; this pushes OS-Atlas-7B from 18.9% to 48.1%, a 154-percentage-point relative gain. **ZoomClick** [53], from Princeton, introduces a training-free three-stage zoom pipeline operating on a 2×2 grid at depth 3 with $0.5\times$ shrinkage per iteration and a minimum crop size of 768 pixels, improving UI-Venus-72B from 61.4% to 73.1%.

More recent methods incorporate learned zoom policies. **AdaZoom-GUI** [46] trains a conditional zoom policy via Group Relative Policy Optimization (GRPO), learning to zoom only on predicted small elements while refining instructions through Qwen3.5; it achieves 76.8% on ScreenSpot-Pro (up from 61.6%), the current state of the art. **GUI-ARP** [23] uses attention-map-guided cropping with adaptive stage control, allowing the agent to decide between one-stage and two-stage pipelines, and reaches 60.8% with a 7B model, surpassing UI-TARS-72B's 38.1% baseline. **LASER** [43] applies Monte Carlo preference optimization to learn a self-evolving crop policy, improving Qwen2.5-VL-7B from 26.8% to 47.5%.

A common limitation unites all of these systems: they address where to crop but not whether to use vision at all. Every method assumes the agent will process a visual crop; none considers that for many interaction steps (reading a label, verifying a click succeeded, understanding a menu) text extraction alone suffices. Our work addresses this gap by making the observation modality itself a decision variable.

2.3 Accessibility Trees vs. Vision

The structured-data versus vision debate for GUI agents has resolved into a clear hierarchy of approaches.

Accessibility-first methods such as DirectShell [24] consume only 50–200 tokens per step, 10–25× cheaper than screenshots, and achieve 85% task success when accessibility (a11y) trees are complete. However, Screen2AX [27], which audited 302 macOS applications, found that only 33% provide complete native accessibility metadata. DirectShell's success rate drops sharply on applications with poor a11y semantics: 27.3% on Booking.com, 35.7% on Google Flights, and near zero on custom canvas-based UIs, games, and applications with non-standard widget toolkits.

Vision-first systems such as UI-TARS [35] and OpenAI Operator [33] offer universal compatibility across arbitrary applications, but at 1,200–5,000 tokens per step and with persistent coordinate-precision issues on small elements.

Hybrid approaches currently deliver the best real-world results. UFO2 [50] (Microsoft) queries Windows UI Automation alongside OmniParser-v2 [26], which combines YOLOv8 detection with Florence-2 captioning to fill gaps where a11y metadata is absent. Agent S2 [1] (Simular) employs a Mixture of Grounding Experts fusing visual, OCR, and structural signals. OmniParser-v2 processes screenshots into structured elements at 0.6s per frame on an A100, yielding a 73% accuracy improvement over vision-only baselines. For browser-specific tasks, a11y-first observation dominates, as DOM and a11y trees are well-structured by design; Browser-Use achieves 89.1% on WebVoyager with a11y-first observation [30].

The field has converged on a practical hierarchy: use structured data when available, fall back to vision otherwise, and fuse both when possible. What remains missing is an agent-controlled interface that implements this principle as a runtime decision rather than a static architectural choice. Our `observe` tool achieves this by using a11y data internally when available while exposing a uniform modality-selection API to the agent.

2.4 Context Compression

The growing context of historical observations is the primary cause of exponential latency degradation in multi-step agent trajectories. The latency overhead documented in Section 1 [48] leaves substantial room for reduction.

Fara-7B [44] (Microsoft) provides direct evidence for our approach: it retains only the last 3 screenshots in history while preserving all text, and completes tasks in approximately 16 steps versus 41 for UI-TARS-1.5-7B. The key finding, that text-based history is load-bearing while screenshot history is not, directly motivates our text-first observation design.

On the compression side, the JetBrains "Complexity Trap" study [3], presented at NeurIPS DL4Code 2025, showed that simple **observation masking** (replacing old tool outputs with placeholders such as `[Previous output: 2,847 tokens, masked]`) halves cost while matching or slightly exceeding LLM-based summarization. In software engineering agent turns where observation dominates token count, a placeholder is often better than a poor summary. **AgentOCR** [25] takes a complementary approach by rendering text history as images, exploiting the higher information density of visual token encoding in VLMs; it retains 99.5% of performance while cutting tokens by over 50%, with the agent selecting its compression rate via RL. **ACON** [6] optimizes natural-language compression guidelines through gradient-free search, achieving 26–54% peak token reduction.

These methods are complementary to our approach: FSP reduces per-step observation cost at the source (by returning text instead of images for most modes), while compression techniques like Fara-7B's sliding window and JetBrains' observation masking reduce the cost of retaining historical observations. Applied together, they compound.

2.5 Bio-Inspired Visual Processing

Our work draws on several threads of biological and bio-inspired vision research, though none have previously been applied to GUI agent perception.

Foveated vision transformers. FovealNet [22] and log-polar Vision Transformers [28] apply spatially varying resolution, high acuity at the fixation center and progressively lower toward the periphery, achieving 30–50% FLOP reduction on ImageNet classification while preserving accuracy. These architectures demonstrate that full-resolution processing of the entire visual field is unnecessary, but they have not been applied to screen understanding or GUI tasks.

Dual-stream (magno/parvo) neural networks. A small literature implements the magnocellular/parvocellular split as parallel branches of a neural network. Choi et al. [**?**] introduce *WhereCNN* (coarse, wide field, motion-biased) and *WhatCNN* (fine, narrow field, color-biased) and show that this split reproduces the functional segregation observed in primate dorsal and ventral pathways. Ibrayev et al. [**?**] propose a two-stream foveation-based active vision model in which a dorsal-style stream selects fixation locations while a ventral-style stream performs recognition, evaluated on classification benchmarks. SUGARL [**?**] trains an active-vision RL agent in Atari with concurrent peripheral and foveal inputs and learns a policy over fixation location. All of these systems operate on passive classification, robotics, or game environments; the modality choice is either internal to the model or an RL-trained continuous parameter. None exposes a discrete peripheral-vs-foveal modality as an agent-facing tool call, and none has been applied to GUI observation.

Predictive coding networks. Salvatori et al. [38] introduced PC-Transformers at ICLR 2024, implementing predictive coding within transformer architectures so that only prediction errors propagate through the network. This maps directly to the concept of change-based observation (processing only what is unexpected after an action) but has not been used for agent observation control. Our `check` mode operationalizes this principle: the agent states its expectation, and the system reports only the prediction error.

Active vision. Descendants of the Recurrent Attention Model (RAM) [29], including Active Vision Transformers [10], learn sequential fixation policies via reinforcement learning, selecting where to look next based on task requirements. GUI-Eyes [5] is the closest existing system to this paradigm in the GUI domain, training perception-tool usage via RL. Our approach differs by providing a fixed menu of biologically motivated modes rather than learning the perception policy from scratch.

Task-conditioned visual encoding. Modern VLMs such as Qwen2.5-VL [45] and InternVL 2.5 [7] allow the text prompt to modulate visual processing from early layers via cross-attention. This is theoretically ideal for GUI agents (encoding "click the Submit button" could attend differently than "read the error message") but remains underexploited in current agent architectures.

The key gap. No prior work combines these insights into a unified interface where the agent explicitly picks among the three streams the primate retina sends to cortex – a cheap grayscale wide-field view, an expensive color foveal crop, and a symbolic text channel – via tool calls. Claude Computer Use's `zoom` [2] is the closest production feature but offers only one fidelity level (full-color crop); GUI-Eyes [5] trains the zoom decision via RL but stays within a single color image modality; dual-stream networks [**?**, **?**] implement the magno/parvo split but not as an agent-selectable tool; SUGARL [**?**] gives an RL agent concurrent peripheral and foveal inputs, not a discrete modality choice. Our `observe` tool is, to our knowledge, the first to expose the magno/parvo/VWFA decomposition as an agent-facing interface for GUI perception, with the agent selecting which stream to sample at each step.

3 FSP: Bio-Efficient Observation Control

Current computer-use agents receive a full screenshot at every step, an architectural default inherited from early vision-language pipelines, not a principled design choice. We replace this with Foveated Screen Perception (FSP), a top-down attention system in which the agent picks which of the three parallel streams the primate retina sends to cortex it wants to sample at each step. The large language model acts as an executive controller, analogous to the prefrontal cortex modulating visual cortex via re-entrant connections, and the three tool modes are functional analogs of the magnocellular, parvocellular, and VWFA streams. The cheap grayscale stream dominates general use; the expensive color foveal crop is requested only when color or fine detail is needed; text extraction is used whenever the relevant information is symbolic.

3.1 Design Principles: Three Retinal Streams

The primate retina does not send one unified signal to cortex. It sends at least three parallel streams, each optimized for a different kind of information and each with a characteristic bandwidth [**?**]. FSP takes this decomposition literally: each tool mode is an agent-facing analog of one retinal stream. Three design decisions follow, each motivated by a specific biological fact.

The magnocellular stream is cheap because it sheds color and resolution. Rods ($\sim$120 million cells) are achromatic (a single photopigment) and heavily multiplexed, with up to 100 rods converging onto a single ganglion cell [**?**]. The magnocellular ganglion cells they feed have large receptive fields, respond transiently to luminance changes, and carry the dominant motion signal to V1. This stream answers wide-field questions cheaply: layout, motion, gross structure. In FSP, the `scan` mode takes exactly this shape: a low-resolution grayscale image of the entire screen, optionally annotated with regions that changed since the last scan (the agent-facing analog of magnocellular transient responses). Grayscale is one channel instead of three; low resolution is fewer pixels than full; the combination drops token cost by $\sim$5–10$\times$ versus a full-color full-resolution frame without meaningful loss of spatial layout or motion information.

The parvocellular stream is expensive, so the fovea is small. Cones ($\sim$6 million cells) carry three photopigments (L, M, S) for color and are packed densely in the $\sim 2^\circ$ fovea with near 1:1 cone-to-ganglion ratios [42, **?**]. The parvocellular stream they feed specializes in high spatial-frequency color-opponent signals, the signal that object recognition and fine discrimination require. The brain pays this cost only at the fovea, a tiny fraction of the visual field. In FSP, the `focus` mode mirrors this: a full-color high-resolution crop at a specified coordinate, typically covering $\sim$1/20th of the screen area. Full fidelity is preserved where it matters; the cost is bounded because the crop is small.

VWFA bypasses pixels for text. A specialized patch of left fusiform cortex, the Visual Word Form Area (VWFA), extracts symbolic text from the ventral stream and emits strings of letters [**?**]. Downstream language cortex operates on strings, not pixels. An OCR call or accessibility-API lookup is functionally equivalent: it takes a region (or skips pixels entirely, in the a11y case) and returns text. Over 70% of human fixation time on screens is spent on textual elements [37]; giving the agent a mode that returns text directly, at $\sim$40 tokens per call, skips parvocellular encoding altogether for the dominant mode of screen interaction.

Top-down selection among streams. A final biological fact ties these together: the brain does not attend to all three streams equally all the time. Prefrontal cortex biases which streams are consulted based on current task goals, via re-entrant connections that modulate V1, V2, and V4 activity before the stimulus is even fully processed [9, 8]. An agent searching for

Table 1: The three observation modes and the retinal streams they mirror. Costs are approximate input tokens per invocation.

Mode	Retinal / cortical stream	Returns	Cost
`read`	VWFA: symbolic text extraction from ventral stream	Text at (x, y) via a11y or OCR	$\sim$40 tok
`scan`	Rods $\rightarrow$ magno $\rightarrow$ dorsal ("where"): wide-field grayscale, motion/change detection	Low-res grayscale image of full screen + change annotations since last scan	$\sim$200 tok
`focus`	Cones $\rightarrow$ parvo $\rightarrow$ ventral ("what"): foveal color, fine detail	High-res full-color crop at (x, y)	$\sim$150 tok

the "Submit" button weights the parvocellular stream near forms; one verifying that a dialog appeared weights the magnocellular stream for motion. FSP implements this directly: the LLM, carrying the task goal in its context, picks the mode.

These principles converge on a single design rule: **the three FSP modes are the three retinal streams.** No fourth mode is needed because the retina does not provide a fourth stream. There is no brain region that outputs a text description of an entire screen; there is no separate "did my action succeed?" module (the magnocellular motion channel already handles that). An earlier iteration of this work had four modes (`glance`, `read`, `check`, `look`); we collapsed them to three precisely because keeping only streams with a genuine biological counterpart forces a simpler and more honest design.

3.2 The Observe Tool

The agent interacts with the screen through a single `observe` tool with three modes, summarized in Table 1. The tool accepts a `mode` parameter and, for `read` and `focus`, optional coordinates and a region radius (see Listing 1).

Mode semantics. `read` extracts text content at and around the specified coordinates via the accessibility tree when available, falling back to OCR on a cropped region; it returns only text, no pixels. `scan` captures the screen, downsamples to low resolution (default 480×270), converts to a single grayscale channel, and returns the resulting image together with a short list of regions that changed since the last scan (computed via perceptual hashing and localized pixel diff, focused on the last action point). This mirrors the magnocellular stream's dual role in wide-field layout perception and transient motion detection. `focus` returns a high-resolution full-color crop around (x, y), optionally annotated with accessibility metadata for any elements inside the region. This is the expensive mode, reserved for questions whose answers are specifically chromatic (color-coded indicators), pictorial (icons, charts), or require sub-pixel spatial precision.

3.3 Agent Loop

The FSP agent follows a three-step cycle in which `scan` brackets the action (orienting before it, verifying after it), `read` handles the dominant case of text comprehension, and `focus` is reserved for the minority of questions that require color or fine visual detail:

1. **Orient (magnocellular).** On arriving at a new screen (task start, navigation, or unexpected state), the agent calls `observe`(`mode`="scan") to sample the grayscale wide-field

Listing 1: The `observe` tool schema exposed to the LLM.

```
OBSERVE_TOOL = {
  "name": "observe",
  "description": (
    "Observe␣the␣screen.␣Pick␣the␣cheapest␣mode␣that␣"
    "answers␣your␣current␣question.␣Three␣modes,␣one␣per␣"
    "retinal␣stream:\n"
    "␣read:␣␣What␣does␣the␣text␣say␣here?␣"
    "␣␣␣␣␣␣␣␣(text␣via␣a11y/OCR,␣~40␣tok)\n"
    "␣scan:␣␣Where␣are␣things?␣What␣just␣changed?␣"
    "␣␣␣␣␣␣␣␣(grayscale␣full␣screen␣+␣diffs,␣~200␣tok)\n"
    "␣focus:␣What␣does␣this␣look␣like␣in␣color␣detail?␣"
    "␣␣␣␣␣␣␣␣(color␣crop␣at␣x,y,␣~150␣tok)"
  ),
  "input_schema": {
    "type": "object",
    "properties": {
      "mode": {
        "type": "string",
        "enum": ["read", "scan", "focus"],
      },
      "center_x": {
        "type": "integer",
        "description": "X␣center␣(read/focus)"
      },
      "center_y": {
        "type": "integer",
        "description": "Y␣center␣(read/focus)"
      },
      "radius": {
        "type": "integer",
        "default": 150,
        "description": "Half-width␣of␣region␣in␣px␣(read/focus)"
      }
    },
    "required": ["mode"]
  }
}
```

stream. This returns a low-resolution grayscale image of the full screen plus annotations of any regions that changed since the last scan (∼200 tokens).

2. **Read (VWFA).** The agent calls `observe`(`mode`="read", x, y) to extract text content near the intended action target: a button label, a menu item, an error message (∼40 tokens).
3. **Act.** The agent executes an action: click, type, scroll, or keyboard shortcut.
4. **Verify (magnocellular again).** The agent calls `observe`(`mode`="scan"), whose change annotations serve as the agent's action-verification signal. Three branches follow:
   - Change matches expectation: proceed to the next action.
   - No change: the action likely failed. Try an alternative.
   - Unexpected change: call `read` near the action point to understand what happened. If text is sufficient, adapt; if color or fine visual detail is needed (icon state, color-coded indicator, chart content), escalate to `focus` (parvocellular).

Two asymmetries mirror the biology. First, `focus` – the color foveal stream – is the exception, not the default: it is called only when `read` and `scan` surface something that text and grayscale cannot resolve. Second, `scan` serves double duty for orientation and for action verification, reflecting the fact that the biological magnocellular stream is simultaneously the wide-field layout channel and the transient change-detection channel. In a typical 50-step task we expect ∼85% of observation calls to be `read` or `scan`, with `focus` reserved for the minority of steps that need color or fine detail.

3.4 Why Four Modes Collapse to Three

An earlier iteration of this work had four modes: `glance` (text-only scene description from a11y or whole-screen OCR), `read` (text at a point), `check` (predictive-coding-style action verification, text-only), and `look` (full-color crop). The current design collapses these to three. Each removal has a specific biological justification.

`glance` → `scan`. The earlier `glance` mode returned a synthesized text description of the whole screen, generated by running OCR or walking the a11y tree and composing a paragraph like "Chrome browsing Gmail. Sidebar left, email list center, reading pane right." This inverts the biology: scene gist in primate vision is extracted from a low-spatial-frequency visual signal in ∼36 ms [14, 41], not from a textual paragraph. Modern VLMs are better at reading a grayscale low-res image than at consuming a synthesized English description of a screen, and there is no retinal circuit that produces such a description. Replacing `glance` with `scan` (a real low-res grayscale image, the signal the brain actually gets) removes a translation step and matches the biology.

`check` → annotations on `scan`. The earlier `check` mode implemented predictive coding as a dedicated tool: the agent stated an expectation, the system captured the screen and reported whether reality matched. In biology, this function is not its own module; it is what the magnocellular stream does after every saccade and motor action [11, 17]. A `scan` after an action returns a grayscale image plus a diff ("Dialog appeared near (1200, 400)"). The agent reads both and decides whether the change matches its expectation, using the same language reasoning it uses for every other decision. Removing `check` as a separate mode eliminates a redundancy with `scan`'s transient-change annotations.

`look` → `focus`. We rename `look` to `focus` to make its role explicit: the parvocellular, foveal, color channel, called when color or fine detail is specifically required. The semantics are unchanged.

The result is a three-mode interface in which every mode corresponds to an actual parallel channel of primate vision, and every removed mode was a redundant or biologically fictitious translation of a function another mode already handles.

3.5 Text-First Perception

Why `read` is first-class. Most human screen interaction is text comprehension: reading labels, menus, error messages, status bars, and form fields [37]. The visual processing involved in reading is a means to an end; the information being extracted is text. A mode that returns text directly, via the accessibility tree or OCR, skips the visual encoding step entirely. In our design, the text modes cover an estimated ∼90% of agent observation steps. `look` is reserved for elements where text extraction is genuinely insufficient: unlabeled icons, color-coded status indicators, charts and graphs, and spatial layout relationships.

Why `check` is semantic, not pixel-level. Biological predictive coding is hierarchical and semantic [36, 11]. Higher cortical areas send predictions downward ("I expect a dialog to appear"); lower areas compare against sensory input and propagate only prediction errors upward. These errors are object-level, not pixel-level: surprise in one feature of an object spreads to make the entire object salient, even at the level of V1 [17]. When predictions are confirmed, neural activity is suppressed, not echoed.

The `check` mode mirrors this directly. The agent states its expectation as a natural-language string (e.g., "a save confirmation dialog should appear"). The system captures the screen, compares against the previous state, and returns a semantic verdict: "Confirmed: dialog appeared

as expected" or "NOT as expected: screen unchanged, the Save button is still visible." The implementation uses perceptual hashing (pHash) for coarse change detection, but the output is always a semantic description, not a diff image.

The `check` mode is anchored to the action point. Eye-tracking studies show that after executing an action, humans fixate near the location where they just acted [16]. Changes distant from the action point are routinely missed. Nielsen Norman Group reports that 40% of error messages go unnoticed when they appear away from the user's focus [31]. FSP's `check` mode focuses its comparison on the region surrounding the last action coordinates, matching this biological bias.

Accessibility data as implementation detail. The accessibility tree has no direct biological analog; the brain does not have a "list all objects" mode. It searches for specific targets (Wolfe's Guided Search [47]) or identifies what is at a fixation point. Making accessibility data a top-level observation mode would encourage dumping hundreds of tokens of structured data per step regardless of need.

Instead, accessibility data is an implementation detail inside each mode. `glance` consults the accessibility tree (when available) to produce its layout description, falling back to screenshot-based OCR when the tree is absent or incomplete. `read` retrieves accessibility text content at the target coordinates, falling back to OCR on a cropped region. `look` annotates its image crop with accessibility metadata for elements within the region. The agent never reasons about whether it is receiving accessibility data or vision-derived data; it asks "what is here?" and the system provides the best available answer through whichever backend is more complete. This fusion approach is consistent with hybrid perception findings: the incomplete state of native accessibility [27] makes vision-based fallbacks essential.

3.6 Memory: Vision for the Present, Text for the Past

The growing token cost of historical observations is a primary driver of latency degradation in multi-step tasks [48]. Two recent findings inform our memory design:

- **Text history outperforms screenshot history.** Fara-7B [44] keeps only the last 3 screenshots but retains all textual step summaries, completing tasks in $\sim$16 steps versus $\sim$41 for a screenshot-heavy baseline. Screenshot history does not improve performance; only text-based history does.
- **Masking beats summarization.** JetBrains [3] found that replacing old tool outputs with placeholders (e.g., `[Previous output: 2,847 tokens, masked]`) halves token cost while matching or slightly exceeding LLM-generated summaries. A placeholder is often better than a bad summary.

FSP implements a `SimpleMemory` module (Listing 2) that follows both principles. A sliding window of size N (default N=5) retains the most recent steps in full detail within the conversation context. Older steps are compressed to single-line text summaries (action taken and observation result) with all images dropped. This yields a context profile where the agent has rich perceptual access to the present (via the `observe` tool) and efficient textual recall of the past (via compressed history), matching the biological asymmetry between active perception and episodic memory.

Listing 2: Sliding-window memory. Recent steps are retained in full; older steps are compressed to one-line text summaries with images dropped.

```
class SimpleMemory:
    """Vision for the present, text for the past."""

    def __init__(self, window: int = 5):
        self.steps: list[dict] = []
        self.window = window

    def add(self, step: int, action: str, obs: str):
        self.steps.append({
            "step": step,
            "action": action,
            "obs": obs,   # text summary, never an image
        })

    def get_context(self) -> str:
        """Older steps: one-line text summaries.
        Recent steps kept in full in conversation."""
        lines = []
        for s in self.steps[:-self.window]:
            lines.append(
                f"Step␣{s['step']}:␣{s['action']}"
                f"␣->␣{s['obs']}"
            )
        return "\n".join(lines)
```

4 Experimental Design

4.1 Factorial Design

The central claim of FSP is that agent-controlled observation improves perception efficiency regardless of the underlying model. A single-model evaluation cannot distinguish whether gains stem from the perception interface itself or from idiosyncratic compatibility with one model's training distribution. We therefore employ a 2×2 factorial design crossing two frontier models with two perception backends:

Table 2: Experimental conditions. Each cell is an independent evaluation run. The result is conclusive if FSP outperforms Standard CU on both rows.

	Standard CU (full screenshot every step)	**FSP** (`observe` tool, 4 modes)
GPT-5.4 (OpenAI)	Baseline A	Treatment A
Opus 4.6 (Anthropic)	Baseline B	Treatment B

Both models see an identically named skill (`computer_use`) with an identical action space (click, type, scroll, keyboard shortcut). Only the perception backend differs: Standard CU returns a full screenshot after every action; FSP exposes the `observe` sub-tool with four modes. The system prompt is shared except for perception-specific usage instructions in the FSP condition. Critically, the model receives no signal indicating which condition it is in; it simply uses the tools it is given.

The result is conclusive if FSP wins on both models. This controls for the objection that the tool interface merely suits one model's training. If both GPT-5.4 and Opus 4.6 benefit, the perception interface itself is better, not model-specific tuning.

4.2 Benchmarks

We evaluate in two phases, ordered by iteration speed and ecological validity.

Phase 1: ScreenSpot-Pro (grounding accuracy). ScreenSpot-Pro [21] is a static dataset of GUI screenshots annotated with click targets spanning desktop, mobile, and web interfaces. It requires no virtual machines and permits rapid iteration. The key difficulty is that targets occupy only 0.07% of image area on average (compared to 2.01% in older benchmarks), making it the canonical test for fine-grained GUI grounding. This is precisely the regime where zoom/crop approaches have demonstrated the largest gains. We run all four conditions (2 models $\times$ 2 perception backends).

Phase 2: OSWorld (end-to-end task completion). OSWorld [49] is the largest computer-use benchmark, comprising full desktop tasks executed in Ubuntu virtual machines (VMware or Docker+KVM). The agent connects via VNC and executes actions through `pyautogui`. OSWorld is the benchmark most cited in analyses of agent observation inefficiency [48], documenting the latency and step-count overhead described in Section 1. We run all four conditions.

4.3 Metrics

Table 3: Metrics collected per experimental condition.

Metric	What it measures
Success rate	Task completion. FSP must not degrade task success relative to Standard CU.
Total input tokens per task	The core efficiency claim: expected 19–33$\times$ reduction under FSP.
Observation tokens only	Isolates perception savings from reasoning tokens.
Steps to completion	Whether FSP reduces wasted observation cycles.
Mode distribution (FSP only)	What the agent actually chooses. How often does it use text-only modes vs. fall back to `look`?
Latency per step	Less context leads to faster inference. Measures the latency benefit of reduced observation tokens.
Grounding accuracy	Click precision on small elements (ScreenSpot-Pro).

4.4 Implementation Details

Standard CU backend. After every action, the system captures a full screenshot via VNC and returns it to the model as part of the tool response. The model sees a single `computer_use` tool whose output always includes the full-resolution screenshot. This mirrors the production behavior of current computer-use systems [2, 33].

FSP backend. After every action, the system captures a screenshot internally but does not send it to the model. Instead, the model calls the `observe` sub-tool with a chosen mode. The system processes the internally held screenshot according to the requested mode and returns the result:

- `check`: Compares the current screenshot against the previous one via perceptual hashing (pHash). If the hash distance is below threshold, returns "Screen unchanged." Otherwise, describes semantic changes near the last action point. Returns text only ($\sim$10–50 tokens).
- `read`: Extracts text content at the specified (x, y) coordinates and radius. Uses the AT-SPI accessibility API when available; falls back to OCR on the cropped region. Returns text only ($\sim$20–100 tokens).

- `glance`: Generates a scene-level description including application name, screen type, layout structure, and overall state. Uses the accessibility tree when available; falls back to OCR-based layout analysis. Returns text only (∼30–50 tokens).
- `look`: Crops a high-resolution region around the specified coordinates and optionally annotates it with accessibility metadata for any elements in the region. This is the only mode that returns an image (∼50–150 tokens).

Shared infrastructure. Both backends share the same action execution layer (`pyautogui` for mouse/keyboard actions), screen capture mechanism (VNC), and accessibility interface (AT-SPI on the Ubuntu VM). The `SimpleMemory` module compresses old steps to one-line text summaries while keeping recent steps in full detail, following the finding of Fara-7B [44] that text-based history outperforms screenshot history.

4.5 Expected Results

We expect FSP to achieve a 19–33× reduction in observation tokens compared to Standard CU. This estimate derives from the token profile of a typical task:

- Most steps use `check` (∼15 tokens) or `read` (∼40 tokens). These are text-only and cover the dominant modes of screen interaction: action verification and text comprehension.
- Occasional `glance` calls on screen transitions (∼40 tokens).
- Rare `look` calls when visual detail is genuinely needed (∼100 tokens). This is the only mode that incurs image token costs.

Under Standard CU, a 50-step task sends approximately 1,300 tokens per screenshot × 50 steps = ∼65,000 observation tokens. Under FSP, the same task uses approximately 2,000–3,500 observation tokens, assuming a weighted mixture heavily dominated by text-only modes. The reduction is further compounded by the latency benefit: smaller per-step context means faster inference, addressing the dominant bottleneck identified by OSWorld-Human [48].

We further expect FSP to maintain or improve grounding accuracy on ScreenSpot-Pro, because the `look` mode provides a targeted high-resolution crop comparable to the zoom mechanisms that have consistently improved grounding in prior work [20, 53, 46, 5].

5 Discussion

5.1 Top-Down Attention as the Current Contribution

FSP implements top-down (goal-directed) attention: the agent's current task drives which observation mode it selects, where it directs its perceptual resources, and what level of detail it requests. This mirrors the prefrontal cortex to V1/V2 feedback pathway in biological vision, where the "executive" (prefrontal cortex) biases early visual processing toward task-relevant features [9, 8]. In FSP, the LLM serves as the executive controller: it maintains the task goal in its context, evaluates what information it needs next, and selects the appropriate observation mode and coordinates.

The key question this work addresses is whether large language models can effectively serve as executive controllers of their own perception, selecting observation mode, coordinates, and level of detail based on task state, rather than passively receiving full screenshots. The GUI-Eyes system [5] demonstrated that a 3B model can be RL-trained to decide whether to invoke crop and zoom tools (improving ScreenSpot-Pro accuracy from 30.2% to 44.8%). FSP extends this principle by providing a richer set of observation modes and testing whether frontier LLMs can use them effectively without RL training on the observation decision itself.

5.2 Limitations

Purely top-down perception. The current FSP design has no mechanism for unexpected environmental events to capture the agent's attention. If an error popup appears in a region the agent is not attending to, it will be missed, analogous to the change blindness observed in human vision when attention is directed elsewhere [39]. Section 5.3 addresses this limitation directly.

Dependence on LLM mode selection quality. FSP's effectiveness depends entirely on the LLM's ability to choose observation modes wisely. An agent that defaults to `look` on every step would negate the token savings; one that over-relies on `check` might miss important screen changes. We mitigate this through system prompt design, but the mode selection policy could be substantially improved via reinforcement learning, following the reward structure of GUI-Eyes [5] ($R = 0.6R_{\text{acc}} + 0.1R_{\text{format}} + 0.3R_{\text{tool}}$).

Accessibility availability. The `read` and `glance` modes perform best when the accessibility tree (AT-SPI on Linux, UI Automation on Windows, AX API on macOS) provides complete semantic information. However, the limited accessibility coverage documented by Screen2AX [27] means many applications lack complete metadata. FSP mitigates this through OCR fallback, but the quality of the text modes degrades on applications with poor accessibility support and visually complex custom UIs.

OCR and accessibility accuracy. The `read` mode returns text extracted via OCR or accessibility APIs, and errors in extraction propagate directly to the agent's understanding. Small text, unusual fonts, overlapping UI elements, and dynamic content (e.g., animations, loading states) can degrade extraction quality. This is a limitation shared with all hybrid perception systems [50, 1].

5.3 Future Work: Toward Bidirectional Attention

The current FSP design is purely top-down: the agent decides what to observe based on its goals, and the environment is passive. But biological visual attention is bidirectional.

Top-down attention (current). Goal-directed. The prefrontal cortex biases visual processing toward task-relevant features via feedback connections to V1, V2, and V4 [9, 8]. "I need to find the Submit button" leads to attending near form elements. This is what FSP implements: the LLM's task representation drives observation mode selection.

Bottom-up attention (future). Stimulus-driven. Salient changes in the environment capture attention regardless of current goals. A sudden popup, a red error flash, motion, or high contrast grabs biological attention via the superior colliculus and pulvinar pathway before top-down processing engages [15, 40]. Bottom-up saliency evolved because environments contain threats and opportunities that do not wait for goal-directed search: a predator in peripheral vision must interrupt whatever the organism is currently attending to.

In the current FSP design, if an error popup appears in a region the agent is not observing, it will miss it entirely. This is the same failure mode that current full-screenshot systems exhibit: they waste tokens on unchanged regions but at least see the popup (whether or not the model attends to it in its reasoning). FSP trades this passive coverage for efficiency, but the biological solution is not to look everywhere; it is to add a saliency monitor.

Proposed saliency monitor. We propose adding a lightweight, continuously running saliency monitor that generates "attention interrupts," flagging regions where unexpected visual changes occurred. This would operate as follows:

1. **Change detection.** After each action (or on a timer), compute a lightweight pixel diff between the current and previous screenshot. This is cheap: no LLM call required, just image comparison.
2. **Saliency classification.** Classify detected changes by saliency: a new dialog window or error popup scores higher than a subtle highlight change or cursor blink. Classification can use simple heuristics (change area, location relative to common popup regions, color contrast) or a small classifier.
3. **Attention interrupt.** Inject a bottom-up alert into the agent's context: `"Unexpected: dialog appeared at (x, y)"` or `"Alert: red error banner visible at top of screen."`
4. **Top-down follow-up.** The agent then uses its existing top-down modes (`read`, `look`) to inspect the flagged region, integrating the unexpected information into its task plan.

This creates a complete bidirectional attention loop:

- **Top-down** (current FSP): agent $\xrightarrow{\texttt{observe}}$ screen. The LLM's goals drive perception.
- **Bottom-up** (proposed): screen $\xrightarrow{\text{saliency monitor}}$ agent. Environmental changes drive perception.

Combined, this mirrors the full biological visual attention pipeline. The dorsal ("where") stream provides spatial saliency maps that flag locations of interest; the ventral ("what") stream provides object recognition at attended locations; both are modulated by prefrontal task goals [12, 8]. The saliency monitor corresponds to the dorsal/collicular pathway, while the existing FSP modes correspond to ventral-stream processing under prefrontal control.

The attention interrupt mechanism also addresses the 40% error-miss rate identified in change blindness studies: error messages that appear away from the user's current focus of attention go unnoticed in 40% of cases [31]. A saliency monitor would catch these environmental signals that top-down attention alone cannot.

Additional future directions.

- **RL-trained mode selection.** Train the observation policy via reinforcement learning, following GUI-Eyes' reward structure [5]. The reward signal would combine task accuracy, token cost, and observation quality, allowing the agent to learn optimal mode selection rather than relying on prompt engineering.
- **`find` mode for guided visual search.** Add a goal-directed search mode where the agent specifies what it is looking for (e.g., "Submit button", "error message") and the system searches via accessibility lookup or cascaded visual search. This maps to Wolfe's Guided Search theory [47], where top-down feature templates bias the saliency map toward target-matching locations.
- **Grouped-action execution.** Agents observe after every single action even when consecutive actions do not change the screen state meaningfully [48]. FSP's decoupling of actions from observations enables grouped execution: the agent can issue multiple actions before calling `observe`, eliminating redundant observation cycles.

- **Task-conditioned visual encoding.** Current VLMs such as Qwen2.5-VL [45] and InternVL 2.5 [7] use cross-attention mechanisms where the text prompt modulates visual processing from early layers. This capability is underexploited in GUI agents: the system could encode a screenshot differently when the task is "click the Submit button" versus "read the error message," producing task-specific visual features at the encoder level rather than relying on post-hoc attention in the LLM's reasoning.

6 Conclusion

We have presented Foveated Screen Perception (FSP), a bio-inspired top-down attention system for computer-use agents. Rather than sending a full screenshot to the model at every step (the universal default in current production systems), FSP exposes an `observe` tool with four modes: `check` (action verification), `read` (text extraction), `glance` (scene orientation), and `look` (high-resolution image crop). Three of the four modes return text only; `look` is the sole mode that returns pixels, reserved for cases where spatial or visual information cannot be captured as text.

The design principle is that the LLM serves as the executive controller of its own perception, selecting observation mode, coordinates, and level of detail based on task state, mirroring the prefrontal cortex's top-down modulation of visual processing in biological vision. Rather than sending everything to the model, FSP lets the model decide what it needs to see.

We evaluate FSP in a 2×2 factorial design crossing two frontier models (GPT-5.4 and Opus 4.6) with two perception backends (Standard CU and FSP) across two benchmarks: ScreenSpot-Pro for grounding accuracy and OSWorld for end-to-end task completion. The cross-model design ensures that any observed gains are attributable to the perception interface rather than model-specific compatibility. We expect 19–33× reduction in observation tokens, driven by the dominance of text-only modes (`check` and `read`) in typical task execution.

The current contribution is purely top-down: the agent's goals drive observation selection. Future work extends FSP toward bidirectional attention by adding a lightweight saliency monitor that detects unexpected environmental changes and injects bottom-up attention interrupts into the agent's context. Combined with RL-trained mode selection, guided visual search, and grouped-action execution, this points toward a complete bio-inspired perception pipeline for computer-use agents, one where the agent sees not everything, but exactly what it needs.

References

[1] Simular AI. Agent S2: Mixture of grounding experts for GUI agents. *arXiv preprint*, 2025. Fuses visual, OCR, and structural signals via Mixture of Grounding Experts.

[2] Anthropic. Computer use (Beta). `https://docs.anthropic.com/en/docs/build-with-claude/computer-use`, 2025. Production computer-use API with zoom action for cropped sub-regions.

[3] Ramakrishna Bairi et al. The complexity trap: Observation masking for software engineering agents. In *NeurIPS Workshop on Deep Learning for Code (DL4Code)*, 2025. JetBrains. Simple observation masking (replacing old outputs with placeholders) halves cost while matching LLM summarization.

[4] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdollahi, Yinheng Li, Yadong Shi, et al. WindowsAgentArena: Evaluating multi-modal OS agents at scale. *arXiv preprint arXiv:2409.08264*, 2024.

[5] Hao Chen et al. GUI-Eyes: RL-trained perception tool selection for GUI agents. *arXiv preprint*, 2026. First system using RL to train zoom decisions; 3B model achieves 44.8% on ScreenSpot-Pro.

[6] Xiang Chen et al. ACON: Optimizing agent context via gradient-free natural language compression. *arXiv preprint*, 2025. 26–54% peak token reduction via optimized compression guidelines.

[7] Zhe Chen et al. InternVL 2.5: Advancing multimodal perception with prompt-modulated vision. *arXiv preprint*, 2025. Cross-attention enables task-conditioned visual encoding.

[8] Maurizio Corbetta and Gordon L. Shulman. Control of goal-directed and stimulus-driven attention in the brain. *Nature Reviews Neuroscience*, 3(3):201–215, 2002.

[9] Robert Desimone and John Duncan. Neural mechanisms of selective visual attention. *Annual Review of Neuroscience*, 18:193–222, 1995.

[10] Mohsen Fayyaz et al. Active vision transformers. *arXiv preprint*, 2022. Sequential fixation policies via reinforcement learning for Vision Transformers.

[11] Karl Friston. A theory of cortical responses. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 360(1456):815–836, 2005.

[12] Melvyn A. Goodale and A. David Milner. Separate visual pathways for perception and action. *Trends in Neurosciences*, 15(1):20–25, 1992.

[13] Google DeepMind. Project Mariner: A multimodal agent for the web. `https://deepmind.google/technologies/project-mariner/`, 2025. Browser agent using Gemini for web task automation.

[14] Michelle R. Greene and Aude Oliva. Recognition of natural scenes from global properties: Seeing the forest without representing the trees. *Cognitive Psychology*, 58(2):137–176, 2009.

[15] Laurent Itti, Christof Koch, and Ernst Niebur. A model of saliency-based visual attention for rapid scene analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 20(11):1254–1259, 1998.

[16] Yue Jiang et al. Understanding GUI interaction through eye tracking: Gaze patterns in desktop tasks. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2025. 84 participants, 10,282 trials across 900 GUIs; identified Guess-Scan-Confirm cycle.

[17] Georg B. Keller and Thomas D. Mrsic-Flogel. Predictive processing: A canonical cortical computation. *Neuron*, 100(2):424–435, 2018.

[18] Kristin Koch, Judith McLean, Ronen Segev, Michael A. Freed, Michael J. Berry, Vijay Balasubramanian, and Peter Sterling. How much the eye tells the brain. *Current Biology*, 16(14):1428–1434, 2006.

[19] Michael F. Land and Mary Hayhoe. In what ways do eye movements contribute to everyday activities? *Vision Research*, 41(25–26):3559–3565, 2001.

[20] Kaixin Li et al. RegionFocus: Iterative multi-aspect-ratio zoom for GUI grounding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025. Improves UI-TARS-72B from 38.1% to 50.2% and Qwen2.5-VL-72B from 47.8% to 61.6% on ScreenSpot-Pro.

[21] Kaixin Li, Ziyang Liu, Qingrui Feng, Shuhao Liu, Yepang Zhong, Shuai Liu, Xiang Benny Liu, Jian Liu, Yuliang Liu, and Xiang Bai. ScreenSpot-Pro: GUI grounding for professional high-resolution computer use. *arXiv preprint arXiv:2504.07981*, 2025. Targets occupy only 0.07% of image area on average.

[22] Zhenyao Li et al. FovealNet: Foveated vision transformers with spatially varying resolution. *arXiv preprint*, 2023. 30–50% FLOP reduction on ImageNet via spatially varying resolution.

[23] Zhenyu Li et al. GUI-ARP: Attention-map-guided cropping with adaptive stage control. *arXiv preprint*, 2025. Reaches 60.8% with 7B model, surpassing UI-TARS-72B's 38.1% baseline.

[24] Haoran Liu et al. DirectShell: Accessibility-first desktop automation. *arXiv preprint*, 2026. Achieves 85% task success using only accessibility-tree text at 50–200 tokens per step.

[25] Yuhao Liu et al. AgentOCR: Compressing text history via visual token encoding. *arXiv preprint*, 2026. Renders text history as images; retains 99.5% performance while cutting tokens by >50%.

[26] Yadong Lu et al. OmniParser-v2: Screen parsing with YOLOv8 and Florence-2. *arXiv preprint*, 2025. Microsoft. Converts screenshots to structured elements at 0.6s/frame on A100; 73% accuracy improvement.

[27] MacPaw. Screen2AX: Auditing accessibility support across macOS applications. *arXiv preprint*, 2025. Audited 302 macOS applications; found only 33% provide complete native accessibility.

[28] Federico Mele et al. Log-polar vision transformers for foveated image recognition. *arXiv preprint*, 2023. Applies log-polar foveation to Vision Transformers.

[29] Volodymyr Mnih, Nicolas Heess, Alex Graves, and Koray Kavukcuoglu. Recurrent models of visual attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

[30] Magnus Müller et al. Browser-Use: Accessibility-first web agent. *arXiv preprint*, 2025. Achieves 89.1% on WebVoyager with accessibility-first observation.

[31] Nielsen Norman Group. Change blindness in user interfaces. `https://www.nngroup.com/articles/change-blindness/`, 2024. Reports that 40% of error messages go unnoticed when they appear away from the user's focus of attention.

[32] Aude Oliva and Antonio Torralba. Modeling the shape of the scene: A holistic representation of the spatial envelope. *International Journal of Computer Vision*, 42(3):145–175, 2001.

[33] OpenAI. Operator: A computer-using agent. `https://openai.com/index/introducing-operator/`, 2025. Production computer-use system using the Computer-Using Agent (CUA) API.

[34] Yujia Qin et al. UI-TARS-2: Advancing automated GUI interaction with scalable reinforcement learning. *arXiv preprint*, 2025.

[35] Yujia Qin et al. UI-TARS: Pioneering automated GUI interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.

[36] Rajesh P. N. Rao and Dana H. Ballard. Predictive coding in the visual cortex: A functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1):79–87, 1999.

[37] Keith Rayner. Eye movements in reading and information processing: 20 years of research. *Psychological Bulletin*, 124(3):372–422, 1998.

[38] Tommaso Salvatori, Yuhang Song, Thomas Lukasiewicz, Rafal Bogacz, and Zhenghua Xu. PC-Transformers: Predictive coding within transformer architectures. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024. Implements predictive coding in transformers; only prediction errors propagate.

[39] Daniel J. Simons and Ronald A. Rensink. Change blindness: Past, present, and future. *Trends in Cognitive Sciences*, 9(1):16–20, 2005.

[40] Jan Theeuwes. Stimulus-driven capture and attentional set: Selective search for color and visual abrupt onsets. *Journal of Experimental Psychology: Human Perception and Performance*, 20(4):799–806, 1994.

[41] Simon Thorpe, Denis Fize, and Catherine Marlot. Speed of processing in the human visual system. *Nature*, 381(6582):520–522, 1996.

[42] Brian A. Wandell, Serge O. Dumoulin, and Alyssa A. Brewer. Visual field maps in human cortex. *Neuron*, 56(2):366–383, 2007.

[43] Hao Wang et al. LASER: Self-evolving crop policy via Monte Carlo preference optimization. *arXiv preprint*, 2025. Improves Qwen2.5-VL-7B from 26.8% to 47.5% on ScreenSpot-Pro.

[44] Junyang Wang et al. FARA-7B: A fast and reliable agent with text-centric observation. *arXiv preprint*, 2025. Microsoft. Retains only last 3 screenshots but keeps all text history; completes tasks in ∼16 steps vs ∼41 for UI-TARS-1.5-7B.

[45] Peng Wang et al. Qwen2.5-VL: A frontier multimodal model with cross-attention visual encoding. *arXiv preprint*, 2025. Text prompt modulates visual processing from early layers via cross-attention.

[46] Zhengyu Wang et al. AdaZoom-GUI: Adaptive conditional zoom for GUI grounding via GRPO. *arXiv preprint*, 2026. Achieves 76.8% on ScreenSpot-Pro via GRPO-trained conditional zoom.

[47] Jeremy M. Wolfe. Guided search 2.0: A revised model of visual search. *Psychonomic Bulletin & Review*, 1(2):202–238, 1994.

[48] Tianbao Xiao, Danyang Chen, Shunyu Zhang, Shunyu Yao, et al. OSWorld-Human: Understanding human strategies in computer-use agent tasks. *Proceedings of the International Conference on Machine Learning (ICML)*, 2025. Found that planning and reflection calls consume 75–94% of total task latency; screenshot-only history does not improve performance; agents take 1.4–2.7× more steps than human-optimal trajectories.

[49] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Deng, Neel Jain, Robert Linden, Iber Iber, Shuyan Wang, and Graham Neubig. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[50] Chaoyun Zhang et al. UFO2: A UI-focused agent for Windows automation. *arXiv preprint*, 2025. Microsoft. Combines Windows UI Automation with OmniParser-v2.

[51] Yifan Zhang et al. ScreenSeeker: Cascaded recursive cropping for GUI grounding. *arXiv preprint*, 2025. Uses GPT-4o planner with OS-Atlas-7B grounding; pushes OS-Atlas-7B from 18.9% to 48.1%.

[52] Boyuan Zheng et al. SeeAct-V: Visual grounding for web agents. *arXiv preprint*, 2025. Full-screenshot visual grounding for web agents.

[53] Yilun Zheng et al. ZoomClick: Training-free three-stage zoom for GUI grounding. *arXiv preprint*, 2025. Princeton. Achieves 73.1% on ScreenSpot-Pro with UI-Venus-72B.